Star Sorter: Product and Technical Spec (Plasmo)

A browser extension that organizes your GitHub starred repositories into GitHub Lists (the folders on the Stars page), with automatic sorting, an editable preview, and rule management.

Version 1.0 spec. Built with Plasmo, React and TypeScript.

1. Overview

1.1 Problem

A heavy GitHub user can have hundreds of starred repos (the author has 542) and only two Lists. Stars pile up in one long, flat page sorted by date. Finding a repo again means scrolling or searching by name. GitHub Lists solve this, but sorting hundreds of repos into them by hand is slow.

1.2 Solution

An extension that lives on the Stars page. It reads all stars through the GitHub GraphQL API, proposes a list for each repo using editable rules (topics, language, keywords), lets the user review and change every suggestion, then creates the lists and files the repos in bulk.

1.3 Goals

- Sort 500+ stars into lists in a few minutes instead of hours.
- Keep the user in control: nothing changes until they press Apply.
- Work directly on the GitHub Stars page with no separate website.
- Store everything locally. No backend, no accounts, no tracking.

1.4 Non-goals for v1

- Syncing rules across devices or browsers.
- AI/LLM based classification (listed as a future feature).
- Editing or deleting stars themselves (unstarring).

- Managing other people's stars or organization stars.
- Mobile browsers.

1.5 Target user

Developers with a large, unorganized star collection who want a fast way to file them. Comfortable creating a personal access token.

2. Tech stack

Area	Choice	Why
Extension framework	Plasmo	Handles Manifest V3, hot reload, content script UI, messaging and builds for Chrome, Edge and Firefox
Language	TypeScript (strict)	Safer API and rules code
UI	React 18	Plasmo's default, works inside content script UI
Styling	Tailwind CSS (via Plasmo's Tailwind support) or plain CSS modules	Styles are injected into a shadow DOM so they cannot clash with GitHub's CSS
Storage	@plasmohq/storage	Wrapper over chrome.storage with React hooks
Messaging	@plasmohq/messaging	Clean content script to background worker calls
API	GitHub GraphQL v4	Only API that exposes Lists
Package manager	pnpm	Matches Plasmo's template
Testing	Vitest for the rules engine, manual end to end on a test account	Rules engine is pure logic and easy to unit test

Bootstrap command: `pnpm create plasmo --with-src star-sorter`.

3. Architecture

3.1 Components

1. **Content script UI (CSUI)** in `src/contents/stars-panel.tsx`. Injects a floating button and a panel into github.com stars pages. Renders inside a shadow DOM through Plasmo's `getStyle`.
2. **Background service worker** in `src/background/`. Owns all network calls to api.github.com so the token never lives in page context and requests are not subject to page CSP. Exposes handlers through @plasmohq/messaging.
3. **Options page** in `src/options.tsx`. Full-size editor for token and rules (JSON editor with validation and a visual rule editor).
4. **Popup** in `src/popup.tsx`. Small status view: token set or not, last run summary, link to open the Stars page and the options page.
5. **Core library** in `src/lib/`. Pure TypeScript with no browser APIs: rules engine, GraphQL query strings, types. Shared by every other component and unit tested.

3.2 Data flow

1. User opens the Stars page. The content script shows the floating button.
2. User clicks Scan. The content script sends a `scan` message to the background worker.
3. The worker pages through all starred repos and all existing lists with their items, then returns a normalized result.
4. The content script runs the rules engine locally and shows the preview table.
5. User edits targets, then clicks Apply. The content script sends an `apply` message with the final plan.
6. The worker creates missing lists, then updates each repo's list memberships one by one, streaming progress back to the UI.
7. UI shows the summary and offers a page refresh.

3.3 Suggested project structure

```
star-sorter/
  src/
    background/
      index.ts
      messages/
```

```
        scan.ts
        apply.ts
        validate-token.ts
    contents/
      stars-panel.tsx
    components/
      Panel.tsx
      PlanTable.tsx
      RulesEditor.tsx
      TokenForm.tsx
      ProgressBar.tsx
    lib/
      github.ts        (GraphQL client + queries)
      rules.ts         (classification engine)
      defaults.ts      (default rules)
      types.ts
      history.ts       (undo log)
    options.tsx
    popup.tsx
    style.css
  assets/icon.png
  package.json
  tsconfig.json
```

3.4 Manifest settings (in package.json)

```
"manifest": {
  "host_permissions": ["https://api.github.com/*"],
  "permissions": ["storage"]
}
```

The content script matches `https://github.com/*`. The panel only shows on the Stars page. No other permissions are needed.

4. Features

F1. Stars page panel

- A floating "Sort stars" button appears only on stars pages (`?tab=stars` and `/stars`).

- GitHub is a single page app, so the script must detect navigation (listen for `turbo:load`, `turbo:render` or observe URL changes) and show or hide the button accordingly.
- Clicking the button opens a panel anchored bottom right. Panel can be closed, and its state is kept while the user stays on the page.
- Supports GitHub dark and light themes (read `data-color-mode` on the html element).

F2. Token management

- Field to paste a classic personal access token with the `user` scope.
- Token is validated on save with a lightweight query (`viewer { login }`). The panel shows the username on success and a clear message on failure.
- Stored in `chrome.storage.local`. Never sent anywhere except api.github.com.
- A Remove token button clears it.
- Help text with a direct link to create the token with the right scope pre-selected.

F3. Full scan

- Fetches every starred repo: id, nameWithOwner, description, primary language, up to 10 topics, star count, last updated date, whether archived.
- Fetches existing lists and the repos inside each one, so already filed repos are known.
- Pages 100 at a time with a progress counter. Handles 5,000+ stars.
- Results are cached in memory for the session. A Rescan button refreshes them.

F4. Rules engine (auto classification)

Each rule describes one target list:

```
type Rule = {
  name: string            // list name, max 32 chars
  description?: string    // list description
  match: {
    topics?: string[]     // exact topic match
    languages?: string[]  // exact primary language match
    keywords?: string[]   // case insensitive substring in name + description
+ topics
    owners?: string[]     // e.g. "microsoft", "google"
    exclude?: string[]    // keywords that block this rule
  }
}
type RuleSet = {
  private: boolean        // create new lists as private
  maxListsPerRepo: number // default 1
  lists: Rule[]           // evaluated top to bottom, first matches win
}
```

Behavior:

- Rules are evaluated in order. A repo takes the first `maxListsPerRepo` matching lists.
- A repo with no match is marked Unsorted and defaults to skip.
- A score based mode (count of matching signals, highest wins) is a stretch goal. v1 ships first match wins.
- A default rule set is bundled (Web3, Claude and prompting, AI agents, DevOps and cloud, architecture, interview and algorithms, frontend, backend and databases, Rust, Go, dev tools, SEO).

F5. Rules editor

- **JSON mode**: textarea or Monaco editor with schema validation and inline error messages.
- **Visual mode** (options page): add, rename, reorder (drag), delete rules. Chip inputs for topics, languages, keywords.
- Import and export rules as a `.json` file.
- Reset to defaults.
- Live counter: "this rule matches 37 of your stars" computed from the last scan.
- Warns if the rule set would create more than 32 lists (GitHub limit).

F6. Preview and manual override

- Table of every repo that will be considered, with columns: repo name (links to the repo), description snippet, language, suggested list, and a dropdown to change it.
- Dropdown contains existing lists, rule lists, a Skip option, and Create new list.
- Filters: All, Suggested, Unsorted, Already in a list. Text search by name or description.
- Group by suggested list with counts, collapsible.
- Bulk actions: select several rows and set one list for all, or skip all.
- By default repos already inside a list are hidden. A checkbox includes them.
- Counter always shows how many moves will happen.

F7. Apply

- Creates every missing list using `createUserList`. Names are matched case insensitively against existing lists to avoid duplicates.
- For each repo, merges the new list with its current memberships and calls `updateUserListsForItem`. This mutation replaces the full set, so merging is mandatory or the repo would be removed from lists it was already in.
- Runs sequentially with a short delay (about 250 ms) between calls to stay under secondary rate limits.
- Progress bar with current repo and counts. A Cancel button stops after the current call.
- Retries on transient errors (502, 503, secondary rate limit) with exponential backoff, up to 3 times. Failed repos are listed at the end with a Retry failed button.
- A final summary: lists created, repos moved, repos failed, repos skipped.

F8. Undo

- Before applying, the extension records each repo's previous list ids in `chrome.storage.local` (the undo log).
- An Undo last run button restores those memberships and optionally deletes lists that this run created and that are now empty.
- Only the most recent run is kept in v1.

F9. List management

- A Lists tab showing existing lists with repo counts.

- Create, rename, edit description, toggle private or public, delete (with confirmation) through `createUserList`, `updateUserList` and `deleteUserList`.
- Shows lists that are nearing the 32 list limit.

F10. Search and bulk move from the page (stretch for v1.1)

- A small "Move to list" menu injected next to each repo on the native stars page, for one off filing while browsing. Marked stretch because GitHub's DOM changes without notice, so injection must be defensive and fail silently.

F11. Popup

- Shows token status, username, last run result, and quick links: Open my Stars, Open options.

F12. Options page

- Token, rules editor (visual and JSON), import and export, theme, and a danger zone (clear all extension data).

5. GitHub API details

All calls go to `POST https://api.github.com/graphql` with header `Authorization: bearer <token>`.

5.1 Read stars

```
query($after: String) {
  viewer {
    starredRepositories(first: 100, after: $after,
      orderBy: {field: STARRED_AT, direction: DESC}) {
      pageInfo { hasNextPage endCursor }
      nodes {
        id nameWithOwner description isArchived stargazerCount
        primaryLanguage { name }
        repositoryTopics(first: 10) { nodes { topic { name } } }
      }
    }
  }
}
```

5.2 Read lists and their items

```
{ viewer { lists(first: 32) { nodes { id name description isPrivate } } } }

query($id: ID!, $after: String) {
  node(id: $id) {
    ... on UserList {
      items(first: 100, after: $after) {
        pageInfo { hasNextPage endCursor }
        nodes { ... on Repository { id } }
      }
    }
  }
}
```

5.3 Write

```
mutation($i: CreateUserListInput!) { createUserList(input: $i) { list { id
name } } }
mutation($i: UpdateUserListsForItemInput!) { updateUserListsForItem(input:
$i) { clientMutationId } }
mutation($i: UpdateUserListInput!) { updateUserList(input: $i) { list { id }
} }
mutation($i: DeleteUserListInput!) { deleteUserList(input: $i) {
clientMutationId } }
```

`UpdateUserListsForItemInput` takes `itemId` (the repo node id) and `listIds` (the complete desired set).

5.4 Constraints to verify early

- Lists are limited to 32 per account.
- Confirm during the first spike which token types can call the list mutations. A classic token with the `user` scope is the assumed baseline. Fine grained tokens may not be supported. Treat this as the first thing to test, before building any UI.
- Respect GraphQL rate limits (5,000 points per hour) and the secondary limits for mutation bursts. Reading 542 stars costs only a handful of points. Writing is the part that needs pacing.

5.5 Authentication decision

v1 uses a personal access token pasted by the user. It is the simplest option and needs no server. OAuth is not recommended for v1 because the standard web flow needs a client secret, and the device flow endpoints do not allow browser cross origin calls, so both would need a small backend. Revisit if the extension is published widely.

6. Requirements

6.1 Functional requirements

ID	Requirement
FR1	Show the panel only on the user's GitHub Stars pages
FR2	Accept, validate, store and remove a personal access token
FR3	Fetch all starred repos and all lists with membership
FR4	Classify repos with an ordered, editable rule set
FR5	Allow the user to override any suggestion before applying
FR6	Create missing lists and file repos without removing existing memberships
FR7	Show progress, allow cancel, retry failures
FR8	Allow undo of the last run
FR9	Allow list create, rename, privacy change and delete
FR10	Import, export and reset rules
FR11	Work in GitHub dark and light themes

6.2 Non functional requirements

ID	Requirement
NFR1	Scan of 500 stars completes in under 15 seconds on a normal connection
NFR2	Preview stays responsive with 5,000 rows (virtualized list)
NFR3	No data leaves the browser except requests to api.github.com
NFR4	Token is never logged, never put in page context, never in URLs
NFR5	UI styles isolated in a shadow DOM and cannot break GitHub's page
NFR6	Extension survives GitHub UI changes: only the stars page URL check and floating button are required for core function
NFR7	Clear, human readable error messages for token, scope, rate limit and network failures
NFR8	Accessible: keyboard navigable, visible focus states, ARIA labels on controls
NFR9	Works on Chrome and Edge (Chromium, Manifest V3). Firefox build is optional through Plasmo's target flag

6.3 Security and privacy

- Only the `storage` permission and the `api.github.com` host permission.
- Token lives in `chrome.storage.local` and is read only by the background worker.
- Never inject remote code. No analytics. No third party requests.
- Treat repo descriptions as untrusted text. Render with React text nodes only, never `dangerouslySetInnerHTML`.
- Privacy policy for store listing states: local only, no collection.

7. UI and user flows

7.1 First run

1. User installs the extension and opens the Stars page.
2. Button appears. Panel opens to the token form with a link explaining how to create the token.
3. After a valid token the panel shows "Connected as <username>" and the Scan button.

7.2 Main flow

1. Scan. Progress text shows counts.
2. Preview opens grouped by suggested list. Unsorted repos sit in their own group at the top so the user can deal with them first.
3. User adjusts dropdowns or edits rules and re-classifies without rescanning.
4. Apply. Progress bar runs. Summary appears with Undo and Refresh page buttons.

7.3 Panel layout

- Header: title, connection status, close.
- Tabs: Sort, Lists, Rules, Settings.
- Sort tab: Scan button, filters, grouped table, Apply bar pinned at the bottom with the move count.

7.4 Edge states

- No token: only the token form is enabled.
- Invalid or expired token: banner with a Fix token action.
- Zero stars or zero matches: friendly empty state pointing to the rules editor.
- Rate limited: show retry time and pause automatically.
- User navigates away during apply: apply continues in the background worker. Panel reconnects to progress when reopened.

8. Data model (local storage)

```
storage: {
  token: string
  rules: RuleSet
  lastScan?: { at: number, starCount: number, listCount: number }
  undoLog?: { at: number, createdListIds: string[],
              previous: Record<repoId, string[]> }   // repo id to previous
list ids
  settings: { includeSorted: boolean, theme: 'auto'|'light'|'dark' }
}
```

Star and list data is held in memory only and is not persisted, because it is cheap to refetch and can go stale.

9. Error handling

Case	Behavior
401 Bad credentials	Clear status, prompt for new token
Missing scope or forbidden mutation	Explain which scope is needed, link to token settings
403 secondary rate limit	Read `retry-after`, pause, resume automatically
5xx or network drop	Backoff and retry up to 3 times, then add to failed list
GraphQL partial errors	Show the message for that repo and continue with the rest
32 list limit reached	Block apply before starting, show which rules to merge

10. Testing plan

- **Unit tests (Vitest)**: rules engine with fixtures for topic, language, keyword, owner, exclude, ordering, maxListsPerRepo, and no match. Merge logic for list memberships. Undo log generation.

- **Mocked API tests**: GraphQL client with pagination, retries and error mapping.
- **Manual end to end on a throwaway GitHub account** with about 50 stars: scan, apply, undo, cancel mid run, expired token, offline.
- **Cross browser smoke test** in Chrome and Edge.
- **SPA navigation test**: move between profile tabs and confirm the button appears and disappears correctly.

11. Milestones

Milestone	Scope	Notes
M0 Spike	Script that creates a list and files one repo with `updateUserListsForItem` using your token	Confirms token scope and mutation behavior before any UI
M1 Core	Plasmo project, background worker, token form, scan, rules engine with defaults, simple preview, apply with merge	Matches the working prototype
M2 Control	Editable dropdowns, filters, grouping, progress, cancel, retry	Main quality of life
M3 Safety	Undo, validation, error mapping, rate limit handling	Needed before real use on all 542 stars
M4 Management	Lists tab, visual rules editor, import and export	
M5 Polish	Light and dark themes, accessibility, icons, options and popup pages	
M6 Release	Privacy policy, store screenshots, Chrome Web Store and Edge Add-ons submission	Optional Firefox build

12. Future ideas

- Optional LLM suggestions for unmatched repos, using the user's own API key stored locally.

- Score based classification and rule suggestions learned from the lists you already filled.
- Per repo "Move to list" menu on the native page (F10).
- Detect stale or archived repos and offer an Archive list.
- Sync rules through `chrome.storage.sync`.
- Keyboard shortcuts and a command palette.
- Export a Markdown or JSON backup of all stars and their lists.

13. Risks and open questions

Risk	Mitigation
List mutations may be restricted to certain token types	M0 spike answers this first
GitHub could change or remove the Lists GraphQL schema (it is a relatively new surface)	Isolate all API calls in `lib/github.ts` and keep types in one place
Secondary rate limits on bulk writes	Sequential calls with delay, backoff, resume
Injected UI breaks on GitHub redesign	Floating button needs no DOM anchors, native menu injection is optional
Wrong auto classification	Preview and per repo override, undo, rules are editable
32 list cap too small for finer organization	Warn early, suggest broader rules, optional use of multi list membership

14. Definition of done for v1

- Fresh install, token entry, scan and apply works on an account with 500+ stars without errors.
- Every suggested list can be overridden before apply.
- Undo restores the previous state of the last run.
- No requests leave the browser except to api.github.com.
- Unit tests pass for the rules engine and merge logic.

- Builds load cleanly in Chrome and Edge with no manifest warnings.